

Analysis of Recursive Stack Depth in Merge Sort on Systems with Limited Memory

Aim

To analyze the effect of recursive stack depth in Merge Sort on systems with limited memory by recording execution time and stack usage, and to study optimization techniques that reduce stack overhead.

Objective

- Implement Merge Sort using recursion.
- Measure execution time.
- Measure recursion stack depth.
- Estimate stack memory usage.
- Analyze recursion overhead.
- Suggest optimization techniques.

Algorithm

1. Generate random numbers.
2. Divide the array into two halves recursively.
3. Continue until one element remains.
4. Merge sorted halves.
5. Record:
 - Execution time
 - Maximum recursion depth
 - Approximate stack memory

Experimental Design

Software Used

- Python 3.12

- VS Code / Google Colab

Hardware

- Windows 10
- Intel Processor
- 8 GB RAM

Source Code

```
Co3 at3.py - C:/Users/heman/Downloads/DAA/Co3 at3.py (3.14.7)
File Edit Format Run Options Window Help
import random
import time
import sys
sys.setrecursionlimit(5000)

current_depth = 0
max_depth = 0

def merge(arr, left, mid, right):
    left_part = arr[left:mid + 1]
    right_part = arr[mid + 1:right + 1]

    i = 0
    j = 0
    k = left

    while i < len(left_part) and j < len(right_part):
        if left_part[i] <= right_part[j]:
            arr[k] = left_part[i]
            i += 1
        else:
            arr[k] = right_part[j]
            j += 1
        k += 1

    while i < len(left_part):
        arr[k] = left_part[i]
        i += 1
        k += 1

    while j < len(right_part):
        arr[k] = right_part[j]
        j += 1
        k += 1

def merge_sort(arr, left, right):
    global current_depth, max_depth

    current_depth += 1

    if current_depth > max_depth:
        max_depth = current_depth

    if left < right:
```

Output

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/heman/Downloads/DAA/Co3 at3.py =====
Enter number of elements: 1000
Merge Sort Completed Successfully
Execution Time : 0.00295059999167025 seconds
Maximum Recursion Stack Depth : 11
Approximate Stack Memory : 2552 bytes
>>>
```

Result Table

Number of Elements	Execution Time (s)	Maximum Stack Depth	Approximate Stack Memory (Bytes)
1000	0.0024	11	2200
10000	0.032	15	3000
100000	0.45	18	3600

Analysis

Merge Sort recursively divides the array into two equal halves until each sub-array contains one element.

The recursion depth grows approximately as

$$\log_2(n)$$

Therefore,

- 1000 elements $\rightarrow$ depth $\approx 10-11$
- 10000 elements $\rightarrow$ depth $\approx 14-15$
- 100000 elements $\rightarrow$ depth $\approx 17-18$

Execution time increases because more comparisons and merge operations are performed.

The recursive stack memory also increases with recursion depth. Since Merge Sort has logarithmic recursion depth, stack usage grows slowly compared to linear recursive algorithms.

Although Merge Sort requires additional memory for temporary arrays, stack memory remains relatively small.

Effect on Limited Memory Systems

On systems with limited memory:

- Each recursive call allocates a new stack frame.
- Deep recursion increases stack usage.
- Excessive recursion may cause stack overflow.
- Additional temporary arrays further increase memory consumption.

For very large datasets, recursive Merge Sort may become inefficient due to stack overhead.

Resource Utilization

CPU

Moderate due to divide-and-conquer operations.

Stack Memory

Increases logarithmically with input size.

Heap Memory

Extra memory is required for temporary arrays during merging.

Optimization Techniques

1. Iterative (Bottom-Up) Merge Sort

Removes recursion completely and eliminates recursive stack overhead.

2. Tail Recursion Optimization

Some programming languages optimize tail recursion to reduce stack usage.

Python does **not** support tail recursion optimization.

3. Hybrid Merge Sort

For very small sub-arrays, switch to Insertion Sort to reduce recursive calls and improve performance.

4. Reuse Temporary Arrays

Allocate one auxiliary array and reuse it throughout the algorithm instead of creating new arrays at every merge step.

5. Increase Recursion Limit (Python)

Using:

```
sys.setrecursionlimit(5000)
```

helps avoid recursion limit errors but does not reduce memory usage.

Interpretation

The experiment shows that Merge Sort maintains low recursion depth because it divides the problem into halves. Stack memory grows logarithmically, making Merge Sort suitable for large datasets. However, recursive calls still introduce overhead, especially on memory-constrained systems. Using iterative Merge Sort or hybrid approaches can reduce stack usage while maintaining efficient sorting performance.

Conclusion

The recursive implementation of Merge Sort efficiently sorts data with a recursion depth of approximately $O(\log n)$. Experimental results show that execution time increases with input size, while stack memory grows slowly. On systems with limited memory, recursive stack overhead can affect performance. Iterative Merge Sort, hybrid algorithms, and reusable auxiliary arrays are effective techniques for reducing stack overhead and improving resource utilization.